UML-Based Modular Designs and Open C Interfaces in Embedded Software-Defined Radio (SDR) Stacks

I. Introduction

The landscape of modern defense and commercial communication systems is increasingly dominated by Software-Defined Radios (SDRs), which represent a paradigm shift from fixed-function hardware to flexible, reconfigurable software-centric platforms.[1] This evolution is driven by the need for rapid waveform prototyping, interoperability, and platform independence, allowing for the quick adaptation to new threats and technological advancements. A critical enabler for these capabilities is the adoption of modular design principles, often formalized through approaches like the Modular Open Systems Approach (MOSA).[1]

MOSA emphasizes the use of widely supported, consensus-based open standards for system interfaces, promoting a structure where components can be incrementally added, removed, or replaced throughout their lifecycle.[3] This strategic approach aims to enhance efficiency, foster competition, and accelerate innovation in acquisition and sustainment. Within the context of SDR, achieving such modularity and openness necessitates robust design methodologies and flexible implementation techniques.

This report explores the application of Unified Modeling Language (UML) for defining modular designs in SDR architectures and examines practical examples of open C code interfaces within embedded SDR stacks. It delves into how high-level design concepts translate into concrete, interoperable C implementations, particularly within frameworks like the Software Communications Architecture (SCA), Sensor Open Systems Architecture (SOSA), and C4ISR/EW Modular Open Suite of Standards (CMOSS). The discussion highlights the critical role of middleware technologies such as CORBA and Data Distribution Service (DDS) in facilitating seamless communication across disparate components and platforms.

II. UML-Based Modular Design in SDR Architectures

Unified Modeling Language (UML) serves as a de-facto standard for graphical notation and modeling in software engineering, providing a comprehensive set of diagrams to visualize, specify, construct, and document software systems.[5] Its utility extends across various stages of system development, from problem space analysis to solution space design and architectural constraint modeling.[7]

A. Unified Modeling Language (UML) Fundamentals for Modular Design

UML diagrams are categorized into structural diagrams, which depict the static elements of a system, and behavioral diagrams, which illustrate dynamic aspects and interactions.[6] For modular design, several UML diagrams are particularly pertinent. Class diagrams, for instance, are static structure diagrams that describe a system's structure by showing classes, their attributes, methods, and the relationships among objects, providing an overview of an application.[10] They are crucial for representing detailed designs and programming constructs.[8]

Component diagrams are another vital structural UML diagram type, illustrating how different parts of a system interact.[6] These diagrams are especially useful for clarifying component interfaces, highlighting dependencies, and understanding the organization and interactions within complex software systems.[12] Components communicate through interfaces, which are visually represented by "lollipops" (provided interfaces) and "sockets" (required interfaces).[13] This visual representation underscores the contractual nature of interfaces, where a component offers a service (lollipop) and another component requires it (socket), enabling clear separation and potential for interchangeability.

Behavioral diagrams, such as state machine diagrams and sequence diagrams, model the dynamic aspects of a system.[6] State machine diagrams show the discrete behavior of a system by depicting state transitions of a part of that system, useful for understanding the internal behavior of a single object.[6] Sequence diagrams, conversely, provide a dynamic illustration of object interactions in chronological order, ideal for modeling multiple objects and the sequence of operations within a system.[7] The choice between these depends on the specific analysis and design task, with

state machine diagrams excelling at single-object internal behavior and sequence diagrams at multi-object interactions.[7]

B. Modular Open Systems Approach (MOSA) and its Alignment with UML

MOSA is an integrated business and technical strategy that employs a modular design using Major System Interfaces (MSI). It is fundamentally about creating a system structure that allows for severable components to be incrementally added, removed, or replaced throughout the lifecycle.[3] A core tenet of MOSA is the decoupling of interface and service implementation, allowing them to follow separate lifecycles and enabling plug-and-play capability.

UML plays a pivotal role in realizing MOSA-aligned SDR systems. By providing a common graphical notation, UML facilitates the specification of modular architectures, ensuring that the design separates systems, subsystems, and components into major functions and elements. The explicit modeling of interfaces using UML diagrams, particularly component diagrams, directly supports MOSA's emphasis on well-defined and widely supported standard interfaces.[3] This allows designers to visualize and manage the "contracts" between components, which is essential for achieving the interoperability and reusability benefits promised by MOSA. The rigorous definition of these interfaces in UML ensures that different modules and systems can effectively communicate and exchange data, which is a primary benefit of MOSA implementation.

C. Software Communications Architecture (SCA) and UML

The Software Communications Architecture (SCA) is a widely adopted standard for specifying SDRs, following a Component Based Development (CBD) paradigm.[2] Its primary goal is to promote software reuse and abstract applications from underlying hardware through standardized interfaces, effectively shielding components from physical hardware details.[15]

UML 2.0 is the de-facto standard for graphical notation and modeling in software engineering, making it a natural fit for SCA.[5] SCA provides an assembly model where

heterogeneous components, implementing particular business functionality, are combined into composites.[5] A composite is the basic unit of deployment in SCA, described in an XML language called SCDL.[16] These composites can be hierarchically constructed, with high-level services implemented by lower-level services.[16]

In SCA, interactions between components occur through service interfaces, with communication realized via message exchanges.[5] Each SCA component exposes its business logic through one or more services.[5] A component offers a service to others and consumes functions from other services via service-oriented interfaces.[16] A simple component typically has one service (an addressable interface with operations) and one reference (a dependency on another component's service).[16] These component services and references are internal; to create an external interface, the component must be deployed within a composite.[16]

UML profiles, such as SCA-UML, are used to specify SCA architectures by applying stereotypes to UML 2.0 metaclasses.[5] For instance, an SCA component can be described by a UML 2.0 component stereotyped by

<<SCAComponent>>.[5] State machines in UML 2.0 can specify the behavior of various model elements, including protocols related to service interfaces or ports.[5] This formalization in UML helps express both structural and behavioral concepts of SCA in a semi-formal way.[5]

The application of UML in SCA highlights a crucial aspect: the explicit definition of interfaces as contracts. This contractual approach, visualized through UML component diagrams, is not merely a documentation exercise but a foundational element that enables the platform independence and interoperability central to SCA. By clearly delineating what a component provides and requires, it becomes possible to swap out implementations without affecting the overall system, fostering a truly modular and open ecosystem. The use of UML for SCA ensures that the complex interactions within a software-defined radio are well-understood and formally specified, which is vital for system integration and evolution.

D. Sensor Open Systems Architecture (SOSA) and UML

The Sensor Open Systems Architecture (SOSA) Consortium aims to provide commonality across platforms for C4ISR systems, integrating disparate efforts like

HOST and CMOSS into more compatible forms.[17] SOSA defines a modular system structure with tight integration within modules for encapsulating functionality and behaviors, coupled with well-defined open interfaces.[17] The standard incorporates specifications for software components, hardware elements, and electrical/mechanical interfaces.[18]

SoaML (Service-oriented architecture Modeling Language), a UML profile, supports the modeling of service-oriented architectures, including the specification of systems of services, individual service interfaces, and service implementations.[19] SoaML enables modelers to build a services architecture model that depicts how participants (people, organizations, or information system components) work together by providing and using services.[19]

In SoaML, a service is defined as a value delivered through a well-defined interface, available to a community.[19] Participants provide or consume services via ports, which are interaction points.[19] A port offering a service may be designated as a

«Service» port, while one consuming a service may be a «Request» port.[19] Services architecture diagrams, a type of SoaML diagram, represent how participants collaborate by providing and consuming services to achieve a specific purpose.[20]

The SOSA Technical Standard, while focusing on physical interconnects and hardware interfaces like OpenVPX and RF Connector Modules (VITA 67.3) [21], also implicitly relies on the principles of service-oriented architecture for its software components. The emphasis on "well-defined interfaces" in SOSA aligns directly with the capabilities of SoaML to formally specify service interactions.[17] The adoption of SoaML for SOSA architectural modeling ensures that the logical and behavioral aspects of service interactions are as rigorously defined as the physical interfaces, promoting true interoperability across the entire system stack, from hardware to software. This comprehensive approach to interface definition, spanning both physical and logical domains, is critical for achieving the plug-and-play capabilities and adaptability sought by SOSA.

E. C4ISR/EW Modular Open Suite of Standards (CMOSS) and UML

The C4ISR/EW Modular Open Suite of Standards (CMOSS) is a U.S. Army initiative defining a suite of layered open architecture standards to address Size, Weight, and

Power (SWaP) requirements and provide commonality across multiple platforms.[22] CMOSS aims to consolidate hardware (e.g., radio boards, GPS, control boards) into single boxes and enable shared resources and real-time swapping of components.[22] This approach dramatically reduces data stovepipes, simplifying hardware sharing between systems and optimizing the integration and utilization of communication, collection, and management information.[22]

CMOSS employs Model-Based Systems Engineering (MBSE), leveraging Unified Modeling Language (UML) or Systems Modeling Language (SysML).[23] MBSE uses graphical representations with a semantic foundation for modeling system requirements, behavior, structure, and parametrics.[23] Tools like CAMEO Systems Modeler (CSM) are used to develop models containing structural diagrams (Block Definition Diagrams and Internal Block Diagrams), state machine diagrams, and activity diagrams.[23]

The integration of MBSE with UML/SysML in CMOSS is a significant step towards formalizing modular designs. By modeling components and their interfaces at various levels of abstraction, CMOSS ensures that the shared hardware and software components can truly interoperate. The use of detailed diagrams, including state machines and activity diagrams, allows for the precise definition of component behaviors and interactions, which is essential for real-time resource sharing and dynamic reconfiguration in complex C4ISR/EW environments.[23] This systematic modeling approach helps to identify and mitigate integration challenges early in the design phase, reducing overall costs and accelerating the deployment of new capabilities.[22]

III. Open C Interfaces in Embedded SDR Code Stacks

In embedded SDR systems, open C interfaces are crucial for achieving modularity, reusability, and maintainability. These interfaces define the contract between different software components, allowing them to interact without needing to know the internal implementation details of each other. The concept of an "interface as a contract" is a foundational principle that spans from high-level UML design down to low-level C implementation and across various middleware solutions. This consistency provides a clear, agreed-upon specification for interaction, enabling independent development and easier integration and replacement of components.

A. Fundamental C Interface Patterns for Modularity

In C, interfaces are typically defined in one or more .h header files for each component, acting as a public contract for the functions and global variables a component offers or requires.[13]

Function Pointers in Structs

A common pattern for creating open interfaces in C, particularly for Hardware Abstraction Layers (HALs) or drivers, involves using typedef struct containing pointers to functions.[25] This approach allows for a form of polymorphism, where different implementations of a driver (e.g., for different SPI hardware) can conform to the same interface structure. An opaque

void* context pointer (often named ctx, self, or this) can be included in the struct to pass instance-specific data to the functions, enabling multiple instances of a driver.[25]

For example, a generic SPI driver interface can be defined as:

C

```
// spi_interface.h
#ifndef SPI_INTERFACE_H
#define SPI_INTERFACE_H

#include <stdint.h>

// Opaque context type for driver instance data
typedef void* SPI_Context_t;

// Define the SPI Driver Interface using a struct of function pointers
typedef struct {
    // Function to initialize the SPI driver
```

```
    void (*init)(SPI_Context_t ctx);
    // Function to write a byte of data
    void (*write)(SPI_Context_t ctx, uint8_t data);
    // Function to read a byte of data
    uint8_t (*read)(SPI_Context_t ctx);
    // Function to set the Chip Select (CS) state
    void (*set_cs)(SPI_Context_t ctx, int state);
} SPI_DriverInterface;

#endif // SPI_INTERFACE_H
```

An implementation for a specific board would then provide the actual functions and link them to an instance of this interface:

C

```
// board_specific_spi.c (Example implementation for a specific board)
#include "spi_interface.h"
#include <stdio.h> // For demonstration purposes

// Internal state for this specific SPI driver instance
typedef struct {
    int spi_bus_id;
    // Add other board-specific configuration here
} BoardSPI_State_t;

static void board_spi_init(SPI_Context_t ctx) {
    BoardSPI_State_t* state = (BoardSPI_State_t*)ctx;
    printf("Board SPI %d: Initializing...\n", state->spi_bus_id);
    // Actual hardware initialization code
}

static void board_spi_write(SPI_Context_t ctx, uint8_t data) {
    BoardSPI_State_t* state = (BoardSPI_State_t*)ctx;
    printf("Board SPI %d: Writing 0x%02X\n", state->spi_bus_id, data);
    // Actual hardware write operation
}
```

```c
static uint8_t board_spi_read(SPI_Context_t ctx) {
    BoardSPI_State_t* state = (BoardSPI_State_t*)ctx;
    printf("Board SPI %d: Reading...\n", state->spi_bus_id);
    // Actual hardware read operation
    return 0xAB; // Dummy read value
}

static void board_spi_set_cs(SPI_Context_t ctx, int state) {
    BoardSPI_State_t* state = (BoardSPI_State_t*)ctx;
    printf("Board SPI %d: Setting CS to %d\n", state->spi_bus_id, state);
    // Actual hardware Chip Select control
}

// Function to get a configured SPI_DriverInterface for a specific board
SPI_DriverInterface get_board_spi_driver(SPI_Context_t ctx) {
    SPI_DriverInterface interface = {
        .init = board_spi_init,
        .write = board_spi_write,
        .read = board_spi_read,
        .set_cs = board_spi_set_cs
    };
    return interface;
}
```

An application can then use this interface without direct knowledge of the underlying driver implementation:

C

```c
// main.c (Application using the interface)
#include "spi_interface.h"
#include <stdlib.h> // For malloc/free
#include <stdio.h>

// Forward declaration for the function that provides the interface
SPI_DriverInterface get_board_spi_driver(SPI_Context_t ctx);
```

```
int main() {
    // Allocate and initialize instance state for SPI bus 0
    BoardSPI_State_t* spi0_state = (BoardSPI_State_t*)malloc(sizeof(BoardSPI_State_t));
    if (spi0_state == NULL) return 1;
    spi0_state->spi_bus_id = 0;

    // Get the interface for SPI bus 0
    SPI_DriverInterface spi0 = get_board_spi_driver(spi0_state);

    // Use the interface
    spi0.init(spi0_state);
    spi0.set_cs(spi0_state, 0); // Assert CS
    spi0.write(spi0_state, 0x12);
    uint8_t data = spi0.read(spi0_state);
    printf("Application received data: 0x%02X\n", data);
    spi0.set_cs(spi0_state, 1); // Deassert CS

    // Clean up
    free(spi0_state);

    return 0;
}
```

Header Files (.h)

Beyond function pointers, the fundamental way interfaces are defined in C is through header files (.h files).[13] These files declare the functions, global variables, and data structures that a component makes available to others (the "offered interface") and those it expects to be provided elsewhere (the "required interface").[13] This clear separation of declaration from definition is a cornerstone of modular programming in C, allowing compilation units to interact based solely on the interface contract.

Object-Oriented Style in C

While C is not inherently an object-oriented language, it can be used to develop interchangeable components in an object-oriented style.[13] This is achieved by using structures that define data members (state) and function pointers (behavior).[13] The structure's definition itself becomes the interface provided by the component, enabling a form of encapsulation and polymorphism, similar to concepts found in C++ or Microsoft's COM technology.[13]

System-Level Interfaces at Runtime

At a higher level of abstraction, a C component can operate as an independent program, offering interfaces at runtime using operating system features.[13] This includes network protocols (e.g., listening to sockets, implementing HTTP), remote function calls, or various Inter-Process Communication (IPC) techniques such as shared memory and mutexes.[13] These interfaces are essentially contracts about what is provided and expected for communication, extending beyond simple function calls to encompass complex system-level interactions.

The concept of "open interface" in C for embedded SDR is not a singular definition but exists on a spectrum. At the lowest level, it implies well-defined function pointer tables or header files for direct hardware interaction (HAL). At higher levels, it involves standardized middleware for inter-component or inter-system communication. The selection of an interface pattern is a critical design decision, balancing performance requirements with the need for abstraction and interoperability. This implies that different "open interfaces" serve distinct purposes within a complex SDR stack, necessitating a nuanced approach to design.

Table III-1: Comparison of C Interface Implementation Patterns for SDR

Pattern	Primary Use Case in SDR	Key Characteristics	Pros	Cons
Function	Hardware	Low-level, direct	Minimal	Manual state

Pointers in Structs	Abstraction Layers (HALs), device drivers, low-level component APIs	function calls, allows runtime polymorphism	overhead, highly flexible, enables driver interchangeability	management, no built-in type safety beyond C, can be complex for large interfaces
Header Files (.h)	Defining component APIs, shared data structures, constants	Compile-time interface definition, explicit declaration of provided/required elements	Simple, widely understood, enforced by compiler	Limited runtime flexibility, no direct support for behavioral contracts or polymorphism
Object-Oriented Style (Structs + Function Pointers)	Interchangeable software modules, complex device models	Mimics OOP concepts (encapsulation, polymorphism) in C	Improved modularity and reusability for complex components	More verbose than C++, requires disciplined coding practices, no native inheritance
System-Level Interfaces (IPC, Network Protocols)	Distributed SDR applications, inter-process communication, waveform deployment	High-level, network-transparent, language/platform agnostic (via protocols)	Enables distributed systems, robust for inter-application communication, supports scalability	Higher overhead (serialization, network latency), more complex setup, requires OS support

B. Hardware Abstraction Layer (HAL) in Embedded SDR

Role of HAL

A Hardware Abstraction Layer (HAL) is a crucial component in embedded systems, particularly for SDRs, as it provides a standard interface that decouples higher-level

software from device-specific hardware intricacies.[26] This abstraction allows hardware vendors to implement unique features without necessitating modifications to the application code above the HAL.[26] In the context of SDR, the HAL is instrumental in enabling reconfigurability; it allows the communication system to be updated or reconfigured by merely changing software or reprogrammable logic, rather than modifying physical hardware.[27]

Examples of HAL in SDR Context

Analog Devices offers practical examples of HAL implementation for SDR. Their Linux IIO (Industrial I/O) drivers and libiio server abstract the low-level details of hardware, providing a simplified programming interface.[27] This interface facilitates real-time data exchange and system control over TCP, allowing complex signal processing algorithms to be developed and executed in higher-level environments like MATLAB or Simulink.[27]

Furthermore, model-based design environments, such as those provided by MathWorks (MATLAB/Simulink with Embedded Coder and HDL Coder), can automatically generate C and HDL code.[27] This generated code, which includes the necessary hardware interface drivers, can then be deployed onto platforms like Xilinx Zynq SoCs, streamlining the transition from simulation to production and reducing manual coding errors.[27] This automated generation of C code for HAL and signal processing layers addresses the challenge of manually writing error-prone code for complex SDR systems. The ability to generate C code from high-level models is a powerful enabler for maintaining open interfaces and ensuring consistency between design and implementation, indicating a future where such tools are central to defining and enforcing open interfaces in generated C code.

Code Example 2: Conceptual C HAL for a Generic Radio Front-End

This conceptual HAL demonstrates how a generic interface can be defined for a radio front-end, allowing different hardware implementations to be swapped without altering the higher-level application logic.

C

```c
// radio_hal.h (Generic Radio HAL Interface)
#ifndef RADIO_HAL_H
#define RADIO_HAL_H

#include <stdint.h>
#include <stdbool.h>

// Opaque context for radio device instance
typedef void* Radio_Handle_t;

// Enum for radio operational modes
typedef enum {
    RADIO_MODE_RX,
    RADIO_MODE_TX,
    RADIO_MODE_OFF
} Radio_Mode_t;

// Structure defining the Radio HAL interface using function pointers
typedef struct {
    // Initialize the radio hardware
    bool (*init)(Radio_Handle_t handle);
    // Deinitialize the radio hardware
    void (*deinit)(Radio_Handle_t handle);
    // Set the center frequency in Hz
    bool (*set_frequency)(Radio_Handle_t handle, uint64_t freq_hz);
    // Set the sample rate in samples per second
    bool (*set_sample_rate)(Radio_Handle_t handle, uint32_t sample_rate_sps);
    // Set the radio's operational mode (RX, TX, OFF)
    bool (*set_mode)(Radio_Handle_t handle, Radio_Mode_t mode);
    // Read I/Q samples from the radio
    int (*read_iq_samples)(Radio_Handle_t handle, int16_t* iq_buffer, uint32_t
num_samples);
    // Write I/Q samples to the radio
    int (*write_iq_samples)(Radio_Handle_t handle, const int16_t* iq_buffer, uint32_t
num_samples);
    // Get current signal strength indicator (e.g., RSSI)
    int32_t (*get_rssi)(Radio_Handle_t handle);
```

```
} Radio_HAL_Interface_t;

#endif // RADIO_HAL_H
```

A specific implementation for a particular radio hardware (e.g., Radio A) would then provide the concrete functions:

C

```c
// specific_radio_a_hal.c (Partial Implementation for Radio A)
#include "radio_hal.h"
#include <stdio.h>

// Internal context for Radio A hardware
typedef struct {
    uint32_t base_address; // Example: hardware register base address
    bool is_initialized;
    uint64_t current_freq;
    //... other Radio A specific registers, internal state, etc.
} RadioA_Context_t;

// Example implementation of the init function for Radio A
static bool radioA_init(Radio_Handle_t handle) {
    RadioA_Context_t* ctx = (RadioA_Context_t*)handle;
    if (ctx == NULL) {
        fprintf(stderr, "Error: RadioA_Context_t handle is NULL.\n");
        return false;
    }
    printf("Radio A: Initializing hardware at 0x%X\n", ctx->base_address);
    // Actual hardware initialization code (e.g., register writes, power-up sequences)
    ctx->is_initialized = true;
    return true;
}

// Placeholder for other function implementations for Radio A
static void radioA_deinit(Radio_Handle_t handle) {
    RadioA_Context_t* ctx = (RadioA_Context_t*)handle;
    printf("Radio A: Deinitializing hardware.\n");
```

```
    // Actual hardware deinitialization code
    ctx->is_initialized = false;
}

static bool radioA_set_frequency(Radio_Handle_t handle, uint64_t freq_hz) {
    RadioA_Context_t* ctx = (RadioA_Context_t*)handle;
    printf("Radio A: Setting frequency to %llu Hz.\n", (unsigned long long)freq_hz);
    ctx->current_freq = freq_hz;
    // Actual hardware frequency tuning
    return true;
}

static bool radioA_set_sample_rate(Radio_Handle_t handle, uint32_t sample_rate_sps) {
    RadioA_Context_t* ctx = (RadioA_Context_t*)handle;
    printf("Radio A: Setting sample rate to %u SPS.\n", sample_rate_sps);
    // Actual hardware sample rate configuration
    return true;
}

static bool radioA_set_mode(Radio_Handle_t handle, Radio_Mode_t mode) {
    RadioA_Context_t* ctx = (RadioA_Context_t*)handle;
    const char* mode_str;
    switch (mode) {
        case RADIO_MODE_RX: mode_str = "RX"; break;
        case RADIO_MODE_TX: mode_str = "TX"; break;
        case RADIO_MODE_OFF: mode_str = "OFF"; break;
        default: mode_str = "UNKNOWN"; break;
    }
    printf("Radio A: Setting mode to %s.\n", mode_str);
    // Actual hardware mode configuration
    return true;
}

static int radioA_read_iq_samples(Radio_Handle_t handle, int16_t* iq_buffer, uint32_t num_samples) {
    RadioA_Context_t* ctx = (RadioA_Context_t*)handle;
    printf("Radio A: Reading %u IQ samples.\n", num_samples);
    // Simulate reading data (e.g., from a hardware buffer or ADC)
    for (uint32_t i = 0; i < num_samples * 2; ++i) { // IQ samples are interleaved (I, Q, I, Q...)
        iq_buffer[i] = (int16_t)(i % 256 - 128); // Dummy data
```

```
    }
    return num_samples;
}

static int radioA_write_iq_samples(Radio_Handle_t handle, const int16_t* iq_buffer, uint32_t
num_samples) {
    RadioA_Context_t* ctx = (RadioA_Context_t*)handle;
    printf("Radio A: Writing %u IQ samples.\n", num_samples);
    // Simulate writing data (e.g., to a hardware DAC or transmit buffer)
    return num_samples;
}

static int32_t radioA_get_rssi(Radio_Handle_t handle) {
    RadioA_Context_t* ctx = (RadioA_Context_t*)handle;
    printf("Radio A: Getting RSSI.\n");
    // Simulate RSSI reading
    return -70; // Dummy RSSI value in dBm
}

// Function to get a configured Radio_HAL_Interface_t for Radio A
Radio_HAL_Interface_t get_radio_a_hal(Radio_Handle_t ctx) {
    Radio_HAL_Interface_t hal = {
        .init = radioA_init,
        .deinit = radioA_deinit,
        .set_frequency = radioA_set_frequency,
        .set_sample_rate = radioA_set_sample_rate,
        .set_mode = radioA_set_mode,
        .read_iq_samples = radioA_read_iq_samples,
        .write_iq_samples = radioA_write_iq_samples,
        .get_rssi = radioA_get_rssi
    };
    return hal;
}
```

Code Example 3: Analysis and Snippets from fm_receiver.c (RTL-SDR Interaction)

The fm_receiver.c program provides a concrete example of C code interacting with an SDR device, specifically an RTL-SDR dongle.[28] This program demonstrates a software-defined wideband FM receiver, showcasing how a low-level hardware interface is utilized for real-time signal processing.

The program directly includes "rtl-sdr.h", which provides the API for controlling and reading data from the RTL-SDR device.[28] Key functions used include

rtlsdr_open for device initialization, rtlsdr_set_sample_rate and rtlsdr_set_center_freq for configuring the radio, and rtlsdr_read_sync for synchronously acquiring raw I/Q samples.[28] This direct interaction represents a highly performant, hardware-specific interface.

The get_samples_rtl_sdr function serves as the bridge between the generic receive processing function and the specific RTL-SDR hardware, reading raw 8-bit I/Q samples into a buffer.[28] The

receive function then processes this buffer. The core FM demodulation is performed by computing the phase difference between successive complex samples using atan2(cimag(product), creal(product)).[28] This is a direct implementation of a digital signal processing algorithm in C, optimized for performance. Furthermore, the program leverages the

fftw3.h library for Fast Fourier Transform (FFT) operations, enabling efficient frequency domain filtering (e.g., low-pass filtering) of the demodulated audio signal.[28]

A notable aspect of the fm_receiver.c design is its use of a function pointer (get_samples_f) for sample acquisition.[28] This allows the

receive function to be agnostic to the data source, capable of processing samples either from a live RTL-SDR device (get_samples_rtl_sdr) or from a recorded file (get_samples_file).[28] This demonstrates a simple yet effective form of interface-based modularity at the function level, enhancing the reusability of the core demodulation logic.

C

```
// From fm_receiver.c [28]
#include "rtl-sdr.h" // Hardware API header
```

```
// Function pointer type for generic sample acquisition
typedef int (*get_samples_f)(uint8_t *buffer, int buffer_len);

// RTL-SDR device handle (global for simplicity in this example)
rtlsdr_dev_t *device = NULL;

// Function to get samples directly from RTL-SDR device
int get_samples_rtl_sdr(uint8_t *buffer, int buffer_len) {
    int len;
    // Synchronous read from the RTL-SDR device
    rtlsdr_read_sync(device, buffer, buffer_len, &len);
    return len == buffer_len; // Return 1 on success, 0 otherwise
}

// Main processing function, designed to accept any function conforming to get_samples_f
void receive(get_samples_f get_samples, int buffer_size) {
    uint8_t buffer; // Buffer for raw I/Q samples
    double complex sample;
    double complex product;
    double complex prev_sample = 0 + 0 * I; // Initialize previous sample for phase difference
calculation
    //... FFT setup (fftw_plan, fft_in, fft_out, filter_mid, filter_mid2)...

    // Continuous loop to acquire and process samples
    while(get_samples(buffer, buffer_size) == 1) {
        // Core FM demodulation loop: compute amplitudes from phase difference
        for (int i = 0; i < buffer_size; i+=2) {
            // Convert raw 8-bit I/Q samples (0-255) to complex numbers centered around 0
            sample = (buffer[i] - 127) + (buffer[i+1] - 127) * I;
            // Compute product of current sample and conjugate of previous sample for phase change
            product = sample * conj(prev_sample);
            // Extract phase angle (FM demodulation)
            fft_in[i/2] = atan2(cimag(product), creal(product));
            prev_sample = sample; // Update previous sample for next iteration
        }
        //... FFT execution and frequency domain filtering...
        //... Downsampling and writing processed audio to standard output...
        // fwrite(output_buffer, sizeof(int16_t), b, stdout);
    }
```

```
}

void main(int argc, char **argv) {
    //... argument parsing for frequency, device index, and optional record file...

    if (record_file!= NULL) {
        // If a record file is specified, use the file-reading interface
        receive(get_samples_file, BUFFER_SIZE);
    } else {
        // Otherwise, open and configure the RTL-SDR device
        if (rtlsdr_get_device_count() - 1 < DEFAULT_DEVICE_INDEX) {
            fprintf(stderr, "Device %d not found\n", DEFAULT_DEVICE_INDEX);
            exit(EXIT_FAILURE);
        }
        if (rtlsdr_open(&device, DEFAULT_DEVICE_INDEX)!= 0) {
            fprintf(stderr, "Error opening device %d: %s", strerror(errno));
            exit(EXIT_FAILURE);
        }
        // Configure RTL-SDR device settings
        rtlsdr_set_tuner_gain_mode(device, 0); // Auto gain
        rtlsdr_set_agc_mode(device, 0);       // RTL2832 auto gain off
        rtlsdr_set_sample_rate(device, DEFAULT_SAMPLE_RATE);
        rtlsdr_set_center_freq(device, DEFAULT_FREQ);
        fprintf(stderr, "Frequency set to %d\n", rtlsdr_get_center_freq(device));
        rtlsdr_reset_buffer(device); // Flush buffer

        // Start receiving and processing data from the RTL-SDR device
        receive(get_samples_rtl_sdr, BUFFER_SIZE);
    }
}
```

C. Middleware for Open Interfaces in SDR

The increasing complexity and distributed nature of modern SDR systems necessitate sophisticated inter-process communication (IPC) mechanisms. Middleware solutions provide higher-level, network-transparent interfaces that abstract away low-level

networking complexities, enabling seamless communication between disparate components and across different platforms. The evolution from simple, low-level IPC to advanced middleware like CORBA and DDS reflects the growing demands for distribution and interoperability in SDR. These middleware technologies are not merely communication channels; they are integral to the definition of "open interfaces," offering standardized mechanisms for discovery, quality of service, and language/platform independence, which are essential for large-scale, heterogeneous SDR deployments.

Common Object Request Broker Architecture (CORBA)

CORBA, a standard defined by the Object Management Group (OMG), is designed to facilitate communication between systems deployed on diverse platforms, operating systems, programming languages, and computing hardware.[29] It employs an object-oriented model, even if the underlying systems are not strictly object-oriented, and is a prime example of a distributed object paradigm.[29] CORBA's strength lies in its ability to enable communication between software written in different languages (e.g., C, C++, Java, Python) and running on different machines, abstracting away implementation details related to specific operating systems, programming languages, and hardware.[29]

Role and IDL: A core component of CORBA is its Interface Definition Language (IDL), which is used to specify the interfaces that objects present to the external world.[29] IDL provides a language-independent definition of SCA interfaces and data types.[30] An IDL compiler translates these IDL interfaces into target-language-specific code (stubs and skeletons), which developers then use to build client and server applications.[29] This IDL-centric approach ensures strong data typing and adherence to method signatures, return types, and exceptions, reducing human errors in distributed communication.[29]

Role in SDR Frameworks (SCA, OSSIE, REDHAWK, OpenRTM-aist): CORBA is extensively utilized as the inter-process communication (IPC) mechanism within various SDR frameworks. The SCA Core Framework, for instance, relies on a CORBA Object Request Broker (ORB) for communication between components.[15] Open Source SCA Implementation – Embedded (OSSIE), an open-source SCA framework, uses CORBA for communications between its components, enabling waveform development where components can be deployed across single or multiple networked

computers.[2] Similarly, REDHAWK, another SDR framework, uses CORBA as part of its core framework to support the development, deployment, and management of real-time software radio applications.[34] OpenRTM-aist, a component framework primarily for robotics, is also built on CORBA technology, allowing components implemented in C++, Java, or Python to communicate seamlessly.[35] The reliance on IDL across different middleware standards like CORBA and DDS highlights a critical strategy for maintaining "openness" in complex software ecosystems. IDL acts as a universal contract language, enabling components implemented in diverse programming languages to interoperate, which is a powerful mechanism for ensuring long-term maintainability, reusability, and vendor independence in SDR development.

Code Example 4: Illustrative CORBA IDL and Conceptual C++ Client/Server Interaction

This example demonstrates how an SDR service might be defined using CORBA IDL, followed by conceptual C++ code snippets for its server-side implementation and client-side usage.

Code snippet

```
// MySDRService.idl (CORBA IDL for an SDR service interface)
module SDR {
    // Interface for controlling radio parameters
    interface RadioControl {
        // Set the center frequency in Hz
        void setFrequency(in long freq_hz);
        // Get the current center frequency in Hz
        long getFrequency();
        // Start the data stream from the radio
        void startStream();
        // Stop the data stream from the radio
        void stopStream();
        // Get a block of processed data (e.g., demodulated audio)
        // 'octet' is CORBA's 8-bit unsigned integer type
```

```
        sequence<octet> getProcessedData(in long max_bytes);
    };

    // Interface for receiving raw IQ data
    interface DataSink {
        // Receive a block of raw IQ data
        void receiveIQData(in sequence<octet> iq_data);
    };
};
```

After compiling this IDL with a CORBA IDL compiler (e.g., omniidl, tao_idl), it would generate C++ header and source files (stubs and skeletons) that define the SDR::RadioControl and SDR::DataSink interfaces in C++.

Conceptual C++ Server-side (Implementation of RadioControl):
This class implements the methods defined in the RadioControl IDL interface.

C++

```
// Conceptual C++ Server-side (implementation of RadioControl)
#include <iostream>
// Include the generated header from IDL compilation (e.g., MySDRService.h or SDR_RadioControl.h)
// For demonstration, we'll assume a simplified header or mock functionality.
// In a real CORBA setup, this would be generated by the IDL compiler.
#include "MySDRService.h" // Placeholder for generated IDL header

// Inherit from the generated POA (Portable Object Adapter) servant base class
class RadioControl_impl : public POA_SDR::RadioControl {
public:
    void setFrequency(CORBA::Long freq_hz) override {
        std::cout << "Server: Setting frequency to " << freq_hz << " Hz\n";
        // Actual hardware control logic here (e.g., interact with HAL)
    }

    CORBA::Long getFrequency() override {
        std::cout << "Server: Getting frequency\n";
        // Return current frequency from hardware or internal state
        return 97700000; // Dummy value for demonstration
    }
```

```
    void startStream() override {
        std::cout << "Server: Starting data stream.\n";
        // Logic to initiate data streaming (e.g., from ADC, to processing chain)
    }

    void stopStream() override {
        std::cout << "Server: Stopping data stream.\n";
        // Logic to halt data streaming
    }

    SDR::RadioControl::octet_seq* getProcessedData(CORBA::Long max_bytes) override {
        std::cout << "Server: Providing up to " << max_bytes << " bytes of processed data.\n";
        // In a real SDR, this would fetch processed audio/baseband data
        SDR::RadioControl::octet_seq* data = new SDR::RadioControl::octet_seq();
        // Example: Populate with dummy data
        data->length(std::min((CORBA::ULong)max_bytes, (CORBA::ULong)5));
        if (data->length() > 0) {
            (*data) = 0x01;
            (*data) = 0x02;
            (*data) = 0x03;
            (*data) = 0x04;
            (*data) = 0x05;
        }
        return data; // Caller is responsible for deleting this sequence
    }
};

// Main server setup (simplified, typically involves ORB initialization and object registration)
/*
int main(int argc, char** argv) {
    try {
        // Initialize the ORB
        CORBA::ORB_var orb = CORBA::ORB_init(argc, argv);

        // Get a reference to the RootPOA and activate the POAManager
        PortableServer::POA_var poa =
PortableServer::POA::_narrow(orb->resolve_initial_references("RootPOA"));
        poa->the_POAManager()->activate();

        // Create an instance of the servant (our implementation)
```

```
        RadioControl_impl* radio_impl = new RadioControl_impl();
        // Activate the servant and get its object ID
        PortableServer::ObjectId_var oid = poa->activate_object(radio_impl);
        // Get the object reference
        CORBA::Object_var obj = poa->id_to_reference(oid.in());

        // Convert the object reference to a string and print it (or register with Naming Service)
        CORBA::String_var sior = orb->object_to_string(obj.in());
        std::cout << "RadioControl service IOR: " << sior.in() << std::endl;

        // Run the ORB, waiting for incoming requests
        orb->run();

        // Clean up
        radio_impl->_remove_ref(); // Release servant reference
        orb->destroy();
    } catch (const CORBA::Exception& ex) {
        std::cerr << "CORBA Exception: " << ex._name() << std::endl;
        return 1;
    }
    return 0;
}
*/
```

Conceptual C++ Client-side:
This code would use the generated stub to interact with the remote RadioControl service.

C++

```
// Conceptual C++ Client-side
#include <iostream>
#include <string>
// Include the generated header from IDL compilation
#include "MySDRService.h" // Placeholder for generated IDL header

// Main client logic (simplified, typically involves ORB initialization and resolving object reference)
/*
int main(int argc, char** argv) {
    try {
        // Initialize the ORB
        CORBA::ORB_var orb = CORBA::ORB_init(argc, argv);

        // Resolve the object reference (e.g., from a file, Naming Service, or IOR string)
        // For example, if the server printed its IOR:
```

```
        // std::string ior_string = "IOR:..."; // Get this from server output or config
        // CORBA::Object_var obj = orb->string_to_object(ior_string.c_str());

        // Or using Naming Service (more typical for discovery)
        // CORBA::Object_var ns_obj = orb->resolve_initial_references("NameService");
        // CosNaming::NamingContext_var name_context = CosNaming::NamingContext::_narrow(ns_obj);
        // CosNaming::Name name;
        // name.length(1);
        // name.id = CORBA::string_dup("RadioControlService");
        // name.kind = CORBA::string_dup("");
        // CORBA::Object_var obj = name_context->resolve(name);

        // Narrow the generic object reference to the specific interface type
        SDR::RadioControl_var radio_control = SDR::RadioControl::_narrow(obj.in());

        if (CORBA::is_nil(radio_control.in())) {
            std::cerr << "Error: Could not narrow object reference to RadioControl." << std::endl;
            return 1;
        }

        // Use the remote object's methods as if they were local
        radio_control->setFrequency(100000000); // Set frequency to 100 MHz
        CORBA::Long current_freq = radio_control->getFrequency();
        std::cout << "Client: Current frequency: " << current_freq << " Hz\n";

        radio_control->startStream();
        SDR::RadioControl::octet_seq_var data = radio_control->getProcessedData(1024);
        std::cout << "Client: Received " << data->length() << " bytes of processed data.\n";
        radio_control->stopStream();

        // Clean up
        orb->destroy();
    } catch (const CORBA::Exception& ex) {
        std::cerr << "CORBA Exception: " << ex._name() << std::endl;
        return 1;
    }
    return 0;
}
*/
```

Data Distribution Service (DDS)

DDS (Data Distribution Service) is another OMG machine-to-machine standard,

specifically designed for dependable, high-performance, interoperable, real-time, and scalable data exchanges using a publish-subscribe pattern.[37] It is particularly well-suited for applications requiring distributed real-time communication, such as aerospace, defense, autonomous vehicles, and robotics.[37] DDS simplifies complex network programming by automatically handling message addressing, data marshalling and demarshalling (allowing different platforms to communicate), delivery, flow control, and retries.[37]

Role and Publish-Subscribe Model: A key advantage of DDS is the decoupling it provides between applications.[37] Applications using DDS do not need explicit knowledge of other participating applications, including their existence or locations, fostering modular and well-structured programs.[37] In this model, nodes that produce information (publishers) create "topics" and publish "samples," which DDS then transparently delivers to subscribers that have declared an interest in those topics.[37]

Topics, QoS, and IDL for Data Types: Users can specify Quality of Service (QoS) parameters to configure discovery and behavior mechanisms upfront, allowing fine-grained control over data delivery characteristics.[37] DDS also leverages an Interface Definition Language (IDL) to define topic data types, ensuring static type checking and extensibility across the DDS global data space.[37] Tools like

eProsima Fast DDS-Gen can automatically generate C++ definitions from these IDL files, streamlining the development process.[39]

DDS provides a C++ API for programming, with several open-source and proprietary implementations available, including eProsima Fast DDS, CycloneDDS, RTI Connext, and OpenDDS.[37] The use of IDL across both CORBA and DDS standards underscores a significant architectural principle: the definition of interfaces and data types in a language-agnostic manner is crucial for achieving true openness and interoperability in complex, heterogeneous systems like SDR. This common approach facilitates the integration of diverse components and ensures long-term system evolution.

Code Example 5: Structural C++ Code for Basic DDS Publisher and Subscriber

This example illustrates the basic structure of a DDS application using eProsima Fast DDS, demonstrating how a data type defined in IDL is used by a publisher to send data and a subscriber to receive it.

First, define the data type in an IDL file (e.g., HelloWorld.idl):

Code snippet

```
// HelloWorld.idl (DDS Topic Definition)
struct HelloWorld
{
    unsigned long index; // A simple counter
    string message;      // A message string
};
```

This IDL file would be processed by eProsima Fast DDS-Gen to generate C++ source files (e.g., HelloWorld.hpp, HelloWorldPubSubTypes.hpp, HelloWorldCdrAux.hpp, etc.) that provide the necessary type definitions and serialization/deserialization logic.

Conceptual C++ Publisher (after IDL compilation):
This code defines a publisher that creates a HelloWorld topic and periodically sends messages.

C++

```
// HelloWorldPublisher.cpp (Simplified for illustration)
#include "HelloWorldPubSubTypes.hpp" // Generated from HelloWorld.idl
#include <fastdds/dds/domain/DomainParticipantFactory.hpp>
#include <fastdds/dds/domain/DomainParticipant.hpp>
#include <fastdds/dds/publisher/Publisher.hpp>
#include <fastdds/dds/topic/Topic.hpp>
#include <fastdds/dds/publisher/DataWriter.hpp>
#include <fastdds/dds/publisher/DataWriterListener.hpp>
#include <iostream>
#include <atomic>
#include <chrono> // For std::chrono::seconds
#include <thread> // For std::this_thread::sleep_for

using namespace eprosima::fastdds::dds;

class HelloWorldPublisher {
private:
```

```
    HelloWorld hello_; // The data object to publish
    DomainParticipant* participant_;
    Publisher* publisher_;
    Topic* topic_;
    DataWriter* writer_;
    TypeSupport type_; // Manages the data type registration

    // Listener for publication events (e.g., when a subscriber matches)
    class PubListener : public DataWriterListener {
    public:
        std::atomic_int matched_count_; // Number of matched subscribers
        void on_publication_matched(DataWriter*, const PublicationMatchedStatus& info) override {
            if (info.current_count_change == 1) {
                matched_count_++;
                std::cout << "Publisher matched a subscriber.\n";
            } else if (info.current_count_change == -1) {
                matched_count_--;
                std::cout << "Publisher unmatched a subscriber.\n";
            }
        }
    } listener_;

public:
    HelloWorldPublisher() : participant_(nullptr), publisher_(nullptr), topic_(nullptr),
writer_(nullptr), type_(new HelloWorldPubSubType()) {}

    ~HelloWorldPublisher() {
        // Clean up DDS entities in reverse order of creation
        if (participant_!= nullptr) {
            if (writer_!= nullptr) {
                publisher_->delete_datawriter(writer_);
            }
            if (publisher_!= nullptr) {
                participant_->delete_publisher(publisher_);
            }
            if (topic_!= nullptr) {
                participant_->delete_topic(topic_);
            }
            DomainParticipantFactory::get_instance()->delete_participant(participant_);
```

```
        }
        delete type_; // Delete the TypeSupport object
    }

    bool init() {
        // Initialize the data to be published
        hello_.index(0);
        hello_.message("Hello DDS World");

        // Create a DomainParticipant (entry point to the DDS domain)
        participant_ = DomainParticipantFactory::get_instance()->create_participant(0,
PARTICIPANT_QOS_DEFAULT);
        if (participant_ == nullptr) return false;

        // Register the data type with the participant
        type_.register_type(participant_);

        // Create a Topic
        topic_ = participant_->create_topic("HelloWorldTopic", "HelloWorld",
TOPIC_QOS_DEFAULT);
        if (topic_ == nullptr) return false;

        // Create a Publisher
        publisher_ = participant_->create_publisher(PUBLISHER_QOS_DEFAULT, nullptr);
        if (publisher_ == nullptr) return false;

        // Create a DataWriter for the topic
        writer_ = publisher_->create_datawriter(topic_, DATAWRITER_QOS_DEFAULT,
&listener_);
        if (writer_ == nullptr) return false;

        return true;
    }

    void publish() {
        if (listener_.matched_count_ > 0) {
            hello_.index(hello_.index() + 1); // Increment message index
            std::cout << "Publishing message: " << hello_.message() << " " << hello_.index() << "\n";
            writer_->write(&hello_); // Send the data
```

```
        } else {
            std::cout << "Waiting for subscribers...\n";
        }
    }

    void run(uint32_t samples_to_send) {
        for (uint32_t i = 0; i < samples_to_send; ++i) {
            publish();
            std::this_thread::sleep_for(std::chrono::seconds(1)); // Wait for 1 second
        }
    }
};

// Example main function for publisher (uncomment to run as standalone)
/*
int main(int argc, char** argv) {
    HelloWorldPublisher mypub;
    if (mypub.init()) {
        mypub.run(10); // Send 10 messages
    }
    return 0;
}
*/
```

Conceptual C++ Subscriber (after IDL compilation):
This code defines a subscriber that listens for HelloWorld messages on the specified topic.

C++

```
// HelloWorldSubscriber.cpp (Simplified for illustration)
#include "HelloWorldPubSubTypes.hpp" // Generated from HelloWorld.idl
#include <fastdds/dds/domain/DomainParticipantFactory.hpp>
#include <fastdds/dds/domain/DomainParticipant.hpp>
#include <fastdds/dds/subscriber/Subscriber.hpp>
#include <fastdds/dds/topic/Topic.hpp>
#include <fastdds/dds/subscriber/DataReader.hpp>
#include <fastdds/dds/subscriber/DataReaderListener.hpp>
#include <fastdds/dds/subscriber/SampleInfo.hpp>
#include <iostream>
#include <atomic>
#include <chrono>
#include <thread>
```

```
using namespace eprosima::fastdds::dds;

class HelloWorldSubscriber {
private:
    DomainParticipant* participant_;
    Subscriber* subscriber_;
    Topic* topic_;
    DataReader* reader_;
    TypeSupport type_;

    // Listener for subscription events (e.g., when a publisher matches, or data is available)
    class SubListener : public DataReaderListener {
    public:
        std::atomic_int matched_count_; // Number of matched publishers
        std::atomic_int samples_received_count_; // Number of samples received
        HelloWorld hello_; // Object to store received data

        SubListener() : matched_count_(0), samples_received_count_(0) {}

        void on_subscription_matched(DataReader*, const SubscriptionMatchedStatus& info) override {
            if (info.current_count_change == 1) {
                matched_count_++;
                std::cout << "Subscriber matched a publisher.\n";
            } else if (info.current_count_change == -1) {
                matched_count_--;
                std::cout << "Subscriber unmatched a publisher.\n";
            }
        }

        void on_data_available(DataReader* reader) override {
            SampleInfo info;
            // Take the next available sample
            if (reader->take_next_sample(&hello_, &info) == ReturnCode_t::RETCODE_OK) {
                // Check if the instance is alive and valid
                if (info.instance_state == ALIVE_INSTANCE_STATE) {
                    samples_received_count_++;
                    std::cout << "Subscriber received: " << hello_.message() << " " << hello_.index()
<< "\n";
```

```
            }
        }
    }
} listener_;

public:
    HelloWorldSubscriber() : participant_(nullptr), subscriber_(nullptr), topic_(nullptr),
reader_(nullptr), type_(new HelloWorldPubSubType()) {}

    ~HelloWorldSubscriber() {
        // Clean up DDS entities in reverse order of creation
        if (participant_!= nullptr) {
            if (reader_!= nullptr) {
                subscriber_->delete_datareader(reader_);
            }
            if (subscriber_!= nullptr) {
                participant_->delete_subscriber(subscriber_);
            }
            if (topic_!= nullptr) {
                participant_->delete_topic(topic_);
            }
            DomainParticipantFactory::get_instance()->delete_participant(participant_);
        }
        delete type_;
    }

    bool init() {
        // Create a DomainParticipant
        participant_ = DomainParticipantFactory::get_instance()->create_participant(0,
PARTICIPANT_QOS_DEFAULT);
        if (participant_ == nullptr) return false;

        // Register the data type
        type_.register_type(participant_);

        // Create a Topic
        topic_ = participant_->create_topic("HelloWorldTopic", "HelloWorld",
TOPIC_QOS_DEFAULT);
        if (topic_ == nullptr) return false;
```

```
        // Create a Subscriber
        subscriber_ = participant_->create_subscriber(SUBSCRIBER_QOS_DEFAULT,
nullptr);
        if (subscriber_ == nullptr) return false;

        // Create a DataReader for the topic
        reader_ = subscriber_->create_datareader(topic_, DATAREADER_QOS_DEFAULT,
&listener_);
        if (reader_ == nullptr) return false;

        return true;
    }

    void run(uint32_t samples_to_receive) {
        // Keep running until the desired number of samples are received
        while (listener_.samples_received_count_ < samples_to_receive) {
            std::this_thread::sleep_for(std::chrono::milliseconds(100)); // Sleep to reduce CPU
usage
        }
        std::cout << "Subscriber finished receiving " << samples_to_receive << " samples.\n";
    }
};

// Example main function for subscriber (uncomment to run as standalone)
/*
int main(int argc, char** argv) {
    HelloWorldSubscriber mysub;
    if (mysub.init()) {
        mysub.run(10); // Expect 10 messages
    }
    return 0;
}
*/
```

IV. Conclusion and Recommendations

The development of sophisticated Software-Defined Radios (SDRs) hinges on robust modular design and the implementation of open interfaces. This report has demonstrated how UML serves as an indispensable tool for defining these modular architectures at a high level, providing a common visual language for specifying components, their relationships, and their interaction contracts. Standards such as SCA, SOSA, and CMOSS leverage UML/SysML to formalize their modular open systems approaches, ensuring interoperability and reusability across complex C4ISR/EW domains.

The translation of these high-level designs into concrete C code requires careful consideration of interface patterns. From fundamental C constructs like function pointers within structs for low-level Hardware Abstraction Layers (HALs) to sophisticated middleware solutions like CORBA and DDS, a spectrum of approaches exists. The choice among these patterns involves a critical balance between performance requirements and the need for abstraction and interoperability. For instance, direct hardware interaction in C, exemplified by the fm_receiver.c using rtl-sdr.h, offers high performance but is hardware-specific. Conversely, middleware like CORBA and DDS provide language- and platform-agnostic communication, crucial for distributed systems, albeit with some overhead.

The consistent reliance on Interface Definition Languages (IDLs) across both CORBA and DDS highlights a powerful strategy for defining open interfaces in a technology-neutral manner. This IDL-centric approach facilitates seamless communication between components implemented in different programming languages, thereby promoting long-term maintainability, reusability, and vendor independence in SDR development. The increasing adoption of model-based design, which can automatically generate C code for HALs and signal processing, further reinforces the ability to maintain consistency between design and implementation, ensuring that the defined open interfaces are accurately reflected in the deployed software.

The analysis reveals that the concept of an "interface as a contract" is a fundamental principle that permeates all layers of SDR development, from abstract UML models to concrete C implementations and middleware specifications. This consistent contractual approach is what truly enables modularity and openness, providing a clear, agreed-upon specification for interaction, regardless of the underlying technology or implementation details. This facilitates independent development, easier integration, and the replacement of components, which are vital for the agile evolution of SDR capabilities.

Recommendations for Best Practices:

1. **Adopt a Model-Based Systems Engineering (MBSE) Approach:** Leverage UML/SysML throughout the SDR development lifecycle to rigorously define modular architectures and their interfaces. This includes using component diagrams for structural relationships, state machine diagrams for behavioral aspects of individual components, and sequence diagrams for inter-component interactions. Consider automated code generation from these models to ensure consistency between design and implementation.
2. **Design Interfaces as Explicit Contracts:** Whether using UML or C code, explicitly define interfaces as clear contracts that specify what is provided and what is expected. This includes precise function signatures, data types, and expected behaviors. This practice minimizes dependencies and promotes a clear separation of concerns, which is essential for component interchangeability.
3. **Implement a Layered Architecture in C:** Structure the embedded SDR code stack into distinct layers, with a well-defined Hardware Abstraction Layer (HAL) at the lowest level to manage hardware-specific interactions. For inter-component or inter-system communication, strategically employ appropriate middleware (e.g., DDS for real-time data flow, CORBA for remote object invocation) to balance performance needs with interoperability requirements.
4. **Leverage Existing Open Standards and Middleware:** Adhere to established open standards such as SCA, SOSA, and MOSA, and utilize mature middleware solutions like CORBA and DDS. These standards and technologies provide proven architectural patterns, robust tools, and foster a broader ecosystem of compatible components, significantly reducing development time and integration challenges.
5. **Prioritize Continuous Integration and Thorough Testing:** Implement continuous integration practices and rigorous testing, particularly for interfaces. This ensures that components conform to their defined contracts and that the overall system remains stable and performs as expected in dynamic SDR environments.
6. **Maintain Comprehensive Documentation:** Provide clear and comprehensive documentation for both UML designs and C code interfaces. This is crucial for facilitating collaboration among development teams, onboarding new engineers, and ensuring the long-term maintainability and evolution of the SDR system.

Works cited

1. MOSA / Open Architecture Systems - SRC, Inc., accessed July 9, 2025, https://www.srcinc.com/products/mosa-open-architecture-systems/

2. Open-Source SCA Implementation-Embedded and Software Communication Architecture, accessed July 9, 2025, https://people.kth.se/~maguire/DEGREE-PROJECT-REPORTS/100215-Edoardo_Paone-with-cover.pdf
3. Implementing a Modular Open Systems Approach in Department of Defense Programs - OUSD(R&E), accessed July 9, 2025, https://www.cto.mil/wp-content/uploads/2025/03/MOSA-Implementation-Guidebook-27Feb2025-Cleared.pdf
4. CLE019 Modular Open Systems Approach (MOSA) - DAU, accessed July 9, 2025, https://www.dau.edu/sites/default/files/Migrated/CopDocuments/CLE019_pf_Mods%200-6.pdf
5. Diagram of an SCA composite [11] | Download Scientific Diagram, accessed July 9, 2025, https://www.researchgate.net/figure/Diagram-of-an-SCA-composite-11_fig1_265114593
6. Learn About All 14 Types of UML Diagrams - Creately, accessed July 9, 2025, https://creately.com/blog/diagrams/uml-diagram-types-examples/
7. UML State Machine Diagrams: Modeling Dynamic System Behavior - Omegapy, accessed July 9, 2025, https://www.alexomegapy.com/post/uml-state-machine-diagrams-modeling-dynamic-system-behavior
8. Diagrams in modeling spaces | Computer Science homework help - SweetStudy, accessed July 9, 2025, https://www.sweetstudy.com/files/diagrams-docx-7415995
9. What is a UML Diagram? | Different Types and Benefits - Miro, accessed July 9, 2025, https://miro.com/diagramming/what-is-a-uml-diagram/
10. UML Class Diagram Tutorial - Visual Paradigm, accessed July 9, 2025, https://www.visual-paradigm.com/guide/uml-unified-modeling-language/uml-class-diagram-tutorial/
11. Create a UML static structure diagram - Microsoft Support, accessed July 9, 2025, https://support.microsoft.com/en-us/office/create-a-uml-static-structure-diagram-bb11b3df-8830-4e31-a437-332585da7ce8
12. 7 Component Diagram Examples: Download and Use Directly, accessed July 9, 2025, https://edrawmax.wondershare.com/development-tips/component-diagram-examples.html
13. C - What Interfaces are in UML Component Diagrams - Stack Overflow, accessed July 9, 2025, https://stackoverflow.com/questions/47723140/c-what-interfaces-are-in-uml-component-diagrams
14. naval open architecture - Secretary of the Navy, accessed July 9, 2025, https://www.secnav.navy.mil/rda/OneSource/Documents/Program%20Assistance%20and%20Tools/Handbooks,%20Guides%20and%20Reports/Page%204/navaloacontractguidebook25oct2007.pdf
15. What is Software Communications Architecture (SCA)? | VIAVI Solutions Inc.,

accessed July 9, 2025, https://www.viavisolutions.com/en-us/what-software-communications-architecture-sca
16. Service Component Architecture (SCA) - IBM, accessed July 9, 2025, https://www.ibm.com/docs/en/cics-ts/6.x?topic=applications-service-component-architecture-sca
17. Understanding SOSA - everything RF, accessed July 9, 2025, https://cdn.everythingrf.com/live/SOSA_ebook_Final_1_638575831347426778_2_3_638761395097370396.pdf
18. Sensor Open Systems Architecture™ / SOSA® Consortium | www.opengroup.org, accessed July 9, 2025, https://www.opengroup.org/sosa
19. Service Oriented Architecture Modeling Language (SoaML) - UML-Diagrams.org, accessed July 9, 2025, https://www.uml-diagrams.org/soaml-uml-profile-diagram-example.html
20. How to Draw SoaML Services Architecture Diagram?, accessed July 9, 2025, https://www.visual-paradigm.com/support/documents/vpuserguide/2535/2539/78362_creatingserv.html
21. Navigating The SOSA Technical Standard | TE Connectivity, accessed July 9, 2025, https://www.te.com/commerce/DocumentDelivery/DDEController?Action=srchrtrv&DocNm=VITA-SOSA-technical-standard&DocType=DS&DocLang=EN
22. What is CMOSS? - REDCOM Laboratories, accessed July 9, 2025, https://www.redcom.com/what-is-cmoss/
23. OUSD(R&E) Review of MOSA Tools and Practices - Office of the Under Secretary of Defense for Research and Engineering, accessed July 9, 2025, https://www.cto.mil/wp-content/uploads/2023/06/MOSA-Tools-2022.pdf
24. C5ISR / EW Modular Open Suite of Standards (CMOSS) Mounted Form Factor (CMFF) Reference Architecture (RA) draft for Industry comment, and Market Research Questionnaire - SAM.gov, accessed July 9, 2025, https://sam.gov/opp/a703e30ee9544dc38b3c8387ea45ca1f/view
25. Writing interfaces in C : r/embedded - Reddit, accessed July 9, 2025, https://www.reddit.com/r/embedded/comments/1fraemy/writing_interfaces_in_c/
26. Hardware abstraction layer (HAL) overview - Android Open Source Project, accessed July 9, 2025, https://source.android.com/docs/core/architecture/hal
27. Using Model-based Design for SDR - Part 1 - Analog Devices, accessed July 9, 2025, https://www.analog.com/en/resources/analog-dialogue/articles/using-model-based-design-sdr-1.html
28. sdr-examples/fm_receiver/fm_receiver.c at master · lgeek/sdr ..., accessed July 9, 2025, https://github.com/lgeek/sdr-examples/blob/master/fm_receiver/fm_receiver.c
29. Common Object Request Broker Architecture - Wikipedia, accessed July 9, 2025, https://en.wikipedia.org/wiki/Common_Object_Request_Broker_Architecture
30. IDL to C++ Language Mapping - Nexus by Hexagon, accessed July 9, 2025, https://nexus.hexagon.com/documentationcenter/en-US/bundle/software-development-kit-2022/page/idl-to-c---language-mapping.html

31. Mapping of OMG IDL to C++ - Oracle Help Center, accessed July 9, 2025, https://docs.oracle.com/cd/E13211_01/wle/wle42/corba/20CPP.PDF
32. README.md - johnngugi/CORBA-Example - GitHub, accessed July 9, 2025, https://github.com/johnngugi/CORBA-Example/blob/master/README.md
33. OSSIE - Gobblerpedia, accessed July 9, 2025, https://gobblerpedia.org/wiki/OSSIE
34. RedhawkSDR/core-framework: REDHAWK is a software ... - GitHub, accessed July 9, 2025, https://github.com/RedhawkSDR/core-framework
35. OpenRTM-aist - Wikipedia, accessed July 9, 2025, https://en.wikipedia.org/wiki/OpenRTM-aist
36. A Software Platform for Component Based RT ... - OpenRTM-aist, accessed July 9, 2025, https://openrtm.org/openrtm/sites/default/files/790/SIMPAR2008_Ando.pdf
37. Data Distribution Service - Wikipedia, accessed July 9, 2025, https://en.wikipedia.org/wiki/Data_Distribution_Service
38. Configure the DDS Interface - MATLAB & Simulink - MathWorks, accessed July 9, 2025, https://www.mathworks.com/help/dds/gs/configure-dds-interface.html
39. 1.3. Writing a simple C++ publisher and subscriber application - 3.2.2, accessed July 9, 2025, https://fast-dds.docs.eprosima.com/en/latest/fastdds/getting_started/simple_app/simple_app.html
40. FastDDS-Based Middleware System for Remote X-Ray Image Classification Using Raspberry Pi - arXiv, accessed July 9, 2025, https://arxiv.org/html/2412.07818v1